

Primeri uz skriptu

Rešeni primeri uz pitanja i odgovore



Numeracija i naslovi prate skriptu, pa se čita uporedo s njom. Ovde su samo primeri za pitanja koja ih u skripti nemaju — definicije se ne ponavljaju.

Preskočena su pitanja 01, 09, 11, 12, 13 i 37, koja već imaju primer ili dijagram, i pitanje 39, koje je spisak sistemskih poziva. Ostalih 38 pitanja je pokriveno.

38

PITANJA SA PRIMEROM

7

CELINA

Sadržaj

01 Konceptualno (ER) modeliranje podataka	4
02 Identifikacioni tip poveznika	4
03 ISA hijerarhija	4
04 Gerund	5
05 Rekurzivni tip poveznika i ER model	6
06 Integritetna komponenta	7
07 Kategorizacija	7
08 Agregacija	7
02 Relacioni model podataka	9
10 Ključ šeme relacije	9
14 Spojevi u relacionom	10
15 Pojam podjezika, operator relacione algebre	10
16 Operacijska komponenta kod relacionog modela	11
03 Fizička organizacija podataka — osnovni pojmovi	12
17 LSO, LSP i FSP — nivoi apstrakcije podataka	12
18 Način memorisanja logičkih veza (MVL) između slogova u LSP	12
19 Klasična organizacija datoteka	13
20 Definicija bloka, strukture i vrste	13
21 Način dodele lokacija slogovima (DLS)	14
22 Serijska datoteka	14
04 Sekvencijalne i indeks-sekvencijalne datoteke	16
23 Traženje sloga u sekvencijalnoj datoteci	16
24 Ažuriranje sekvencijalne datoteke u režimu direktne obrade	16
25 Ažuriranje sekvencijalne datoteke u redoslednom režimu obrade	17
26 Sekvencijalna u redoslednoj obradi	17
27 Građenje indeksa kod indeks-sekvencijalne datoteke	18
28 Kako se formira zona indeksa	18
05 Rasute (hash) datoteke	20
29 Metoda centralnih cifara kvadrata ključa	20
30 Metoda preklapanja kod statički rasutih datoteka	20
31 Metoda ostatka pri deljenju	21
32 Smeštanje prekoračilaca kod statički rasute datoteke	21
33 Rasuta datoteka i B-stablo	22
06 Katalog i sistemske tabele datoteka	24
34 Katalog i upravljanje katalogom	24
35 Sistemska i alokaciona tabela datoteka	24

36	Tabela otvorenih datoteka (TOD)	25
07	Sistemske pozive za rad sa datotekama	26
38	Sistemske pozive	26
40–45	Create, Open, Read, Write, Seek, Close — zajednički primer	26
	Sitne ispravke uz skriptu	29

01

Konceptualno (ER) modeliranje podataka

02 Identifikacioni tip poveznika

Zgrada i stan: broj stana je jedinstven samo unutar jedne zgrade, pa se stan ne može identifikovati bez zgrade.

```
ZGRADA (ŠIFZ, ADRESA)
  | (0,N)
  ◇ SADRŽI          ← identifikacioni tip poveznika
  | (1,1)          ← min = max = 1 prema slabom TE
STAN (BRSTANA, POVRŠINA)
```

Ključ slabog TE nastaje spajanjem ključa nadređenog TE i sopstvenog delimičnog ključa:

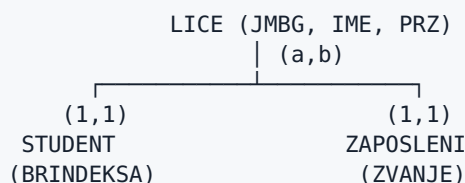
```
ZGRADA({ŠIFZ, ADRESA}, Key = ŠIFZ)
STAN  ({ŠIFZ, BRSTANA, POVRŠINA}, Key = ŠIFZ + BRSTANA)
```

ŠIFZ	BRSTANA	POVRŠINA
Z1	12	54
Z1	13	61
Z2	12	47

Stan 12 postoji dva puta — razlikuju ih tek ŠIFZ + BRSTANA.

Suprotan slučaj (neidentifikacioni TP). Radnik pripada odeljenju s kardinalitetom (1,1), dakle egzistencijalno zavisi od njega, ali ima sopstveni ključ MBR i ne pozajmljuje ključ odeljenja. TP **RADI_U** je zato neidentifikacioni.

03 ISA hijerarhija



Kardinalitet na strani superklase određuje vrstu hijerarhije:

(a,b)	vrsta	šta znači na podacima
(1,1)	totalna, nepresečna	svako lice je tačno jedno: ili student ili zaposleni
(0,1)	parcijalna, nepresečna	ima lica koja nisu ni jedno ni drugo (npr. penzioner)
(1,N)	totalna, presečna	svako lice je bar jedno, može biti i oboje
(0,N)	parcijalna, presečna	student demonstrator je i student i zaposleni, a gost predavač ni jedno

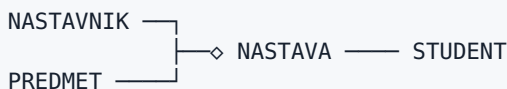
Nasleđivanje: **STUDENT** uz BRINDEKSA vidi i JMBG, IME, PRZ iz **LICE**. Potklasa je id-zavisna od superklase, pa se u relacionom modelu prevodi kao:

```
LICE      ({JMBG, IME, PRZ}, Key = JMBG)
STUDENT  ({JMBG, BRINDEKSA}, Key = JMBG),  STUDENT[JMBG] ⊆ LICE[JMBG]
ZAPOSLENI({JMBG, ZVANJE}, Key = JMBG),  ZAPOSLENI[JMBG] ⊆ LICE[JMBG]
```

04 Gerund

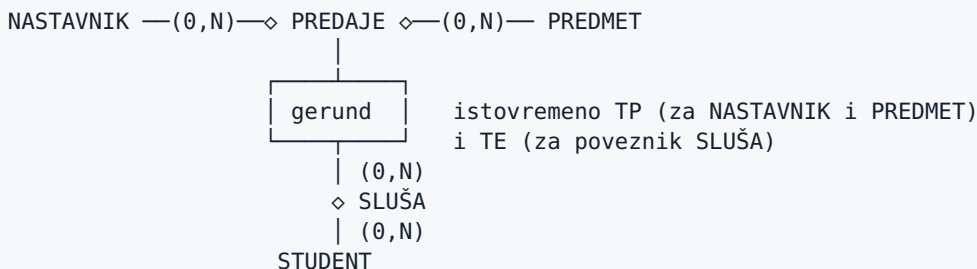
Nastavnik predaje predmet, a student sluša predmet **kod određenog nastavnika** — ne kod bilo kog. To je tačno situacija iz definicije: proizvoljne kombinacije pojava ne smeju biti sadržane u pojavi posmatranog tipa poveznika.

Loše rešenje — ternarni tip poveznika:



Ovakav TP dozvoljava pojavu (Petrović, Baze podataka, Marić) čak i ako Petrović uopšte ne predaje Baze podataka. Pravilo „student može slušati predmet samo kod nastavnika koji ga predaje” nije izrazivo.

Rešenje — gerund: tip poveznika **PREDAJE** se transformiše u tip entiteta i u toj ulozi se povezuje sa **STUDENT**.



Prevođenje u relacioni model:

```
PREDAJE({ŠIFN, ŠIFP}, Key = ŠIFN + ŠIFP)
SLUŠA  ({ŠIFN, ŠIFP, BRINDEKSA}, Key = ŠIFN + ŠIFP + BRINDEKSA)

SLUŠA[ŠIFN, ŠIFP] ⊆ PREDAJE[ŠIFN, ŠIFP]      ← ovo pravilo ternarni TP nije mogao
SLUŠA[BRINDEKSA] ⊆ STUDENT[BRINDEKSA]
```

Na podacima:

PREDAJE

ŠIFN	ŠIFP
N1	P1
N1	P2
N2	P2

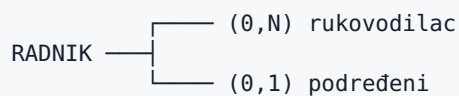
SLUŠA

ŠIFN	ŠIFP	BRINDEKSA
N1	P1	2021/0123
N2	P2	2021/0123

Torka (N2, P1, 2021/0123) se ne može uneti — para (N2, P1) nema u **PREDAJE**.

05 Rekurzivni tip poveznika i ER model

Radnik rukovodi radnicima — obe uloge su iz iste klase entiteta.



Minimalni kardinalitet ne sme biti 1 ni na jednoj strani: kad bi svaki radnik morao imati rukovodioca, lanac bi se zatvorio u ciklus i nijedna pojava ne bi mogla nastati prva.

Kolizija: šema relacije ne sme sadržati dva ista obeležja u različitim ulogama, pa se ključ preimenuje.

```

RADNIK({MBR, IME, MBR_RUK}, Key = MBR)
RADNIK[MBR_RUK] ⊆ RADNIK[MBR]
  
```

MBR	IME	MBR_RUK
101	Ana	NULL
102	Marko	101
103	Iva	101
104	Nikola	102

Ana je direktor (nema rukovodioca), Nikola odgovara Marku, Marko Ani.

Kod veze M:N (npr. sastavnica: deo se sastoji od delova) preimenovani ključ ne staje u istu šemu, pa se TP prevodi u zasebnu šemu:

```

SASTAVNICA({ŠIFD_CELINA, ŠIFD_DEO, KOLIČINA}, Key = ŠIFD_CELINA + ŠIFD_DEO)
  
```

06 Integritetna komponenta

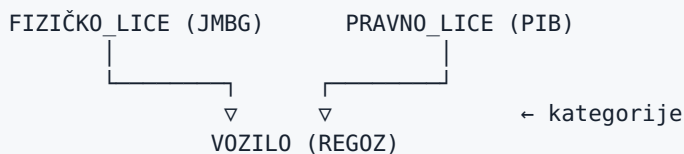
Za svaki tip ograničenja po jedan primer nad šemom **RADNIK**:

tip ograničenja	primer
ograničenje domena	<code>PLATA ∈ Decimal(10,2), PLATA ≥ 0 ; POL ∈ {'M', 'Ž'}</code>
ograničenje pojave tipa	svaka pojava TE RADNIK mora imati vrednost za IME i PRZ
kardinalitet TP	RADNIK $—(0,N)—\diamond$ ANGAŽOVANJE $\diamond—(1,N)—$ PROJEKAT — projekat bez ijednog radnika ne postoji
ograničenje ključa	<code>Key(RADNIK, MBR)</code> — MBR jedinstven i različit od nule

Pravilo za izvođenje zaključaka: iz **RADNIK** $—(1,1)—\diamond$ **RADI_U** $\diamond—$ **ODELJENJE** sledi da je **RADNIK** egzistencijalno zavisan od **ODELJENJE**, što nigde nije zapisano kao zasebno ograničenje nego se izvodi iz kardinaliteta.

07 Kategorizacija

Vozilo je registrovano ili na fizičko ili na pravno lice — nikad na oba, i te dve klase nemaju zajedničkog nadtipa.



- **VOZILO** se povezuje sa **dve** kategorije (minimum je dve),
- svaka pojava pripada **najviše jednoj** — vozilo je ili nečije lično ili firmino,
- **nema id-zavisnosti**: **REGOZ** je sopstveni ključ vozila, ne pozajmljuje se od kategorije.

Po tome se razlikuje od ISA hijerarhije:

	ISA hijerarhija	kategorizacija
broj nadređenih	jedan (superklasa)	više kategorija
ključ	potklasa nasleđuje ključ superklase	tip entiteta ima sopstveni ključ
id-zavisnost	postoji	ne postoji

08 Agregacija

Cela ER struktura „kupac naručuje artikal” posmatra se kao jedan tip entiteta i tek se tako povezuje sa otpremnicom.



Agregat je ovde i korisnički pogled: prodaja vidi jedan pojam „narudžbina”, a ne tri odvojena.

Gerund je najjednostavniji slučaj agregacije — agregira tačno jedan tip poveznika (**PREDAJE** iz pitanja 04), dok agregacija obuhvata proizvoljno složenu ER strukturu.

02

Relacioni model podataka

Primeri u ovoj celini koriste iste dve relacije:

RADNIK

MBR	IME	ŠIFO	PLATA
101	Ana	10	95000
102	Marko	20	72000
103	Iva	NULL	88000

ODELJENJE

ŠIFO	NAZIV
10	Razvoj
20	Prodaja
30	Nabavka

10 Ključ šeme relacije

`RADNIK({MBR, JMBG, IME, PRZ, ŠIFO}, 0)`

- kandidati za ključ: `MBR` i `JMBG` — oba jedinstveno identifikuju torku, pa su **ekvivalentni ključevi**,
- za primarni ključ bira se jedan, npr. `Kp(RADNIK) = MBR`,
- `MBR` i `JMBG` su **primarna obeležja**, `IME`, `PRZ`, `ŠIFO` su **neprimarna**.

Minimalnost se vidi na složenom ključu:

`ANGAŽOVANJE({MBR, ŠIFP, BROJ_SATI}, Key = MBR + ŠIFP)`

MBR	ŠIFP	BROJ_SATI
101	P1	120
101	P2	40
102	P1	80

- `MBR` sam nije ključ — 101 se ponavlja,
- `ŠIFP` sam nije ključ — P1 se ponavlja,
- `{MBR, ŠIFP}` jeste, i to minimalan,

- {MBR, ŠIFO, BROJ_SATI} jedinstveno identifikuje torku, ali **nije ključ** jer nije minimalan — to je nadključ.

14 Spojevi u relacionom

Nad relacijama RADNIK i ODELJENJE odozgo, zajedničko obeležje je ŠIFO.

Prirodan spoj — samo torke sa jednakim vrednostima zajedničkog obeležja:

MBR	IME	ŠIFO	PLATA	NAZIV
101	Ana	10	95000	Razvoj
102	Marko	20	72000	Prodaja

Iva otpada (ŠIFO je NULL), Nabavka otpada (nema radnika).

Dekartov proizvod — sve kombinacije, $3 \times 3 = 9$ torki: (101, Razvoj), (101, Prodaja), (101, Nabavka), (102, Razvoj), ... Rezultat najčešće nema smisla sam po sebi, služi kao međukorak.

Theta spajanje — selekcija iz Dekartovog proizvoda po uslovu, npr. `RADNIK.ŠIFO > ODELJENJE.ŠIFO`:

MBR	IME	ŠIFO	ŠIFO (ODELJENJE)	NAZIV
102	Marko	20	10	Razvoj

Levi spoljni spoj — zadržava sve torke leve relacije:

MBR	IME	ŠIFO	NAZIV
101	Ana	10	Razvoj
102	Marko	20	Prodaja
103	Iva	NULL	NULL

Desni spoljni spoj — zadržava sve torke desne relacije, pa se pojavljuje Nabavka sa NULL vrednostima za radnika.

15 Pojam podjezika, operator relacione algebre

Skupovni operatori zahtevaju uniju-kompatibilne relacije (isti broj i tipovi obeležja).

STUDENT_INF

BRINDEKSA	IME
2021/0100	Ana
2021/0110	Marko

STUDENT_MAT

BRINDEKSA	IME
2021/0110	Marko
2021/0125	Iva

operacija	oznaka	rezultat
unija	STUDENT_INF $\cup$ STUDENT_MAT	0100 Ana, 0110 Marko, 0125 Iva — Marko se ne ponavlja
presek	STUDENT_INF $\cap$ STUDENT_MAT	0110 Marko — sluša oba smera
razlika	STUDENT_INF $-$ STUDENT_MAT	0100 Ana — samo informatika
razlika (obrnuto)	STUDENT_MAT $-$ STUDENT_INF	0125 Iva

Razlika nije komutativna, unija i presek jesu.

16 Operacijska komponenta kod relacionog modela

Projekcija i selekcija nad relacijom **RADNIK**:

```

 $\pi_{\{IME, PLATA\}}(RADNIK)$            $\rightarrow$  (Ana, 95000), (Marko, 72000), (Iva, 88000)
 $\sigma_{\{PLATA > 80000\}}(RADNIK)$      $\rightarrow$  cela torka 101 i cela torka 103
 $\pi_{\{IME\}}(\sigma_{\{PLATA > 80000\}}(RADNIK))$   $\rightarrow$  Ana, Iva

```

Projekcija bira kolone, selekcija bira vrste.

Po jezicima:

```

-- DDL: upravljanje šemom BP
CREATE TABLE RADNIK (MBR INT PRIMARY KEY, IME VARCHAR(30), PLATA DECIMAL(10,2));
ALTER TABLE RADNIK ADD COLUMN ŠIFO INT;
DROP TABLE RADNIK;

-- DML: ažuriranje relacija
INSERT INTO RADNIK VALUES (104, 'Nikola', 65000); -- Add
UPDATE RADNIK SET PLATA = PLATA * 1.1 WHERE MBR = 104; -- Update
DELETE FROM RADNIK WHERE MBR = 104; -- Delete

-- QL: upit nad jednom ili više relacija
SELECT IME FROM RADNIK WHERE PLATA > 80000;

```

Proceduralna vs. specifikaciona. Isti zadatak — „nađi radnike sa platom preko 80000“:

- proceduralno (definiše se šta i kako): otvori datoteku, učitaj prvi slog, ako je PLATA > 80000 prihvati ga, pomeri indikator aktuelnosti na naredni slog, ponavljaj do kraja — program bira jedan po jedan objekat,
- specifikaciono (definiše se samo šta): `SELECT * FROM RADNIK WHERE PLATA > 80000` — redosled i način pristupa bira SUBP.

03

Fizička organizacija podataka — osnovni pojmovi

17 LSO, LSP i FSP — nivoi apstrakcije podataka

Isti podaci na tri nivoa:

LSO — struktura nad tipovima entiteta, poveznika i njihovih obeležja. Ne sadrži nijednu konkretnu vrednost:

```
RADNIK(MBR, IME, PLATA) —◇ RADU ◇— ODELJENJE(ŠIFO, NAZIV)
```

LSP — konkretni podaci u granicama koje LSO propisuje. LSO je model za LSP:

```
(101, Ana, 95000) → (10, Razvoj)
(102, Marko, 72000) → (20, Prodaja)
```

FSP — ta ista LSP smeštena na medijum, sa blokovima, faktorom blokiranja i adresama:

```
blok 0 (rel. adresa 0): [101|Ana|95000][102|Marko|72000][103|Iva|88000][slobodno]
blok 1 (rel. adresa 1): [104|Nikola|65000][...]
```

Jedna LSP može imati više različitih FSP — isti slogovi mogu biti u serijskoj, sekvencijalnoj ili rasutoj datoteci, a da se LSO i LSP ne promene.

18 Način memorisanja logičkih veza (MVL) između slogova u LSP

Logički redosled: 101 → 102 → 103.

a) Fizičkim pozicioniranjem — logički susedni slogovi su i fizički susedni:

```
lokacija:  0      1      2
sadržaj:  [101]  [102]  [103]
```

Naredni slog se dobija uvećanjem adrese za 1. Upis novog sloga između 101 i 102 zahteva pomeranje svih narednih slogova.

b) Pomoću pokazivača kao relativnih adresa — svaki slog pamti adresu narednog:

```
lokacija:  0      1      2
sadržaj:  [101 | →2]  [103 | →/]  [102 | →1]
```

Fizički redosled je 101, 103, 102, ali se logički lanac čita 101 → 102 → 103. Upis novog sloga menja samo dva pokazivača, bez pomeranja podataka.

c) Logičke veze se ne memorišu — u FSP nema nijednog podatka o logičkom susedstvu:

```
lokacija:  0      1      2
sadržaj:  [103]  [101]  [102]
```

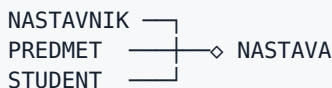
Redosled 101 → 102 → 103 postoji samo kao zahtev korisnika i generiše se posebnim programom (sortiranjem) u trenutku kada zatreba.

19 Klasična organizacija datoteka

Vrste obeležja na jednoj pojavi TE **RADNIK**:

vrsta	primer
elementarno	IME = 'Ana' — ne razlaže se dalje
složeno	ADRESA = (Novi Sad, Bulevar oslobođenja, 12) — razlaže se na Grad, Ulica, Broj
skupovno	TELEFONI = {021/555-111, 064/123-4567} — skup vrednosti istog tipa

N-arni tip poveznika, red $n = 3$:



Kardinalitet se određuje za svaki povezani tip posebno: jedan nastavnik predaje više predmeta za više studenata, jedan student sluša više predmeta kod više nastavnika.

Nedostaci na konkretnom slučaju. Kadrovska služba vodi datoteku **RADNICI.DAT**, računovodstvo svoju **PLATE.DAT**, a obe sadrže ime i adresu radnika:

```
RADNICI.DAT: 101 | Ana | Bulevar oslobođenja 12 | ...
PLATE.DAT:   101 | Ana | Bulevar oslobođenja 12 | 95000 | ...
```

- **redundantnost** — adresa je upisana dva puta,
- **nepovezanost aplikacija** — Ana prijavi selidbu kadrovskoj, adresa se promeni samo u **RADNICI.DAT**, a računovodstvo i dalje šalje obračun na staru adresu (nekonzistentnost),
- **čvrsta povezanost programa i datoteka** — dodavanje polja **EMAIL** u **RADNICI.DAT** menja dužinu sloga, pa se svi programi koji čitaju tu datoteku moraju prekompajlirati.

20 Definicija bloka, strukture i vrste

Slog je 200 B, blok 4 KB:

```
f = [4096 / 200] = 20 slogova po bloku      (faktor blokiranja)
N = 1000 slogova → B = [1000 / 20] = 50 blokova
```

Opšta struktura bloka:

zaglavlje	slog 1	slog 2	...	slog f	slobodan deo
-----------	--------	--------	-----	--------	--------------

Zaglavlje nosi rednu/relativnu adresu bloka, broj slogova i po potrebi pokazivač na naredni blok.

Slogovi konstantne dužine — pozicija i-tog sloga se izračunava:

$\text{pomak} = \text{dužina_zaglavlja} + (i - 1) \times 200.$

Slogovi promenljive dužine — pozicija se ne može izračunati, pa blok nosi direktorijum pomaka i slogovi se pune s jednog, a direktorijum s drugog kraja:

zaglavlje	slog 1	slog 2	slobodno	[p2][p1] broj=2
-----------	--------	--------	----------	-----------------

Blok je niz od 2^n fizičkih blokova: ako je fizički blok 512 B, blok od 4 KB je niz od $2^3 = 8$ fizičkih blokova.

21 Način dodele lokacija slogovima (DLS)

Upisuje se slog sa ključem 35 u datoteku koja sadrži 10, 20, 30, 40, 50.

1. Na kraj datoteke — nova lokacija je fizički susedna poslednjem slogu:

```
pre:  [10][20][30][40][50][ ]
posle: [10][20][30][40][50][35]
```

Upis je jedan pristup, ali datoteka ostaje neuređena → **serijska datoteka**.

2. Prva slobodna lokacija iz spregnute liste slobodnih lokacija — obrisani slogovi ostavljaju rupe koje se lančaju:

```
pre:  [10][20][ ← slobodna, →5 ][40][50][ slobodna, →/ ]
posle: [10][20][35][40][50][ slobodna, →/ ]
```

Prostor obrisanih slogova se ponovo koristi.

3. Adresa kao funkcija vrednosti ključa — $A = f(35)$, npr. $35 \bmod 7 = 0$:

```
baket 0: [35]    ← ide direktno na izračunatu adresu
baket 1: ...
```

Pristup slogu je bez traženja, u jednom koraku → **rasuta datoteka**.

22 Serijska datoteka

Datoteka ima $N = 1000$ slogova, faktor blokiranja $f = 20$:

$B = \lceil (N + 1) / f \rceil = \lceil 1001 / 20 \rceil = \lceil 50,05 \rceil = 51$ bloka

(+1 je specijalan slog — oznaka kraja datoteke.)

Traženje slučajno odabranog sloga, metodom linearnog traženja:

ishod	broj pristupa
uspešno, najbolji slučaj	$R_u = 1$ (slog je u prvom bloku)
uspešno, prosečno	$R_u \approx (B + 1) / 2 = 26$
uspešno, najgori slučaj	$R_u = B = 51$
neuspešno	$R_n = B = 51$ — uvek se pročita cela datoteka

Neuspešno traženje uvek košta ceo prolaz jer slogovi nisu uređeni po ključu, pa se ne može ranije zaključiti da sloga nema. Zato upis (kojem prethodi neuspešno traženje) košta $51 + 1$ pristupa.

04

Sekvencijalne i indeks-sekvencijalne datoteke

23 Traženje sloga u sekvencijalnoj datoteci

Datoteka je uređena po ključu, $N = 1000$ slogova.

Traženje slučajno odabranog sloga (ključ 640) ima smisla samo ako je cela datoteka u OM:

metoda	broj poređenja	kako radi
linearno	prosečno 500	10, 20, 30, ... dok se ne naiđe na 640
binarno	$\lceil \log_2 1000 \rceil \approx 10$	500 → 750 → 625 → 687 → ...

Traženje logički narednog sloga ide isključivo linearno, počevši od tekućeg sloga, i staje u tri slučaja:

```
tekući slog: 30, argument traženja: 35
  učitaj 40 → 40 > 35 → STOP: argument je postao manji od ključa sloga
                    → traženje neuspešno, mesto zaustavljanja je slog 40
```

- traženi slog je pronađen → uspešno,
- argument je postao manji od vrednosti ključa → neuspešno, ali se odmah zna gde slog treba upisati (zato uređenost štedi posao u odnosu na serijsku datoteku),
- došlo se do kraja datoteke → neuspešno.

Ako je datoteka na eksternoj memoriji i velika, traženje slučajno odabranog sloga nema praktičnog smisla — svaki korak binarnog traženja je novi pristup disku. Zbog toga postoji indeks-sekvencijalna organizacija (pitanja 27 i 28).

24 Ažuriranje sekvencijalne datoteke u režimu direktne obrade

Datoteka je u OM, lokacije 0–4:

```
lokacija:  0    1    2    3    4    5
ključ:    10   20   30   40   50   -
```

Upis sloga 35 — prethodi mu jedno neuspešno traženje:

```
1. traženje 35 → zaustavlja se na 40 (lokacija 3), neuspešno
2. pomeranje udesno:  10  20  30  -  40  50
3. upis:              10  20  30  35  40  50
```

Prosečno se pomera polovina slogova, $N/2 = 500$ za datoteku od 1000 slogova. Zato je ovaj režim primenljiv samo dok je datoteka u OM.

Brisanje sloga 20 — prethodi mu jedno uspešno traženje:


```

logičko brisanje (uobičajeno): 10  [20 obrisan]  30  40  50
                                ↑ menja se samo status sloga
fizičko brisanje:             10  30  40  50  —
                                ↑ pomeranje ulevo, prosečno N/2 slogova

```

Modifikacija sloga 30 — takođe uz jedno uspešno traženje; ako se dužina sloga ne menja, sadržaj se prepisuje na mestu, bez ikakvog pomeranja.

25 Ažuriranje sekvencijalne datoteke u redoslednom režimu obrade

Ne menja se postojeća datoteka — gradi se potpuno nova. Ulazne datoteke moraju biti uređene po istom ključu.

```

Ds (stara sekvencijalna):  10  20  30  40  50
Dp (datoteka promena):    15:D  20:B  30:M  40:D  60:B
                           (D = dodavanje, B = brisanje, M = modifikacija)

```

Tok obrade — porede se ključevi tekućih slogova $Ss(Ds)$ i $Sp(Dp)$:

korak	Ss	Sp	poređenje	akcija
1	10	15:D	$Ss < Sp$	prepiši 10 u Dn, čitaj naredni Ss
2	20	15:D	$Ss > Sp$	ključa 15 nema u Ds → upiši 15 u Dn, čitaj naredni Sp
3	20	20:B	$Ss = Sp$	brisanje → ne upisuj ništa, čitaj naredni Ss i Sp
4	30	30:M	$Ss = Sp$	modifikacija → upiši izmenjeni 30' u Dn, čitaj naredni Ss i Sp
5	40	40:D	$Ss = Sp$	dodavanje postojećeg → greška u Dg, upiši 40 nepromenjen u Dn
6	50	60:B	$Ss < Sp$	prepiši 50 u Dn, čitaj naredni Ss → kraj Ds
7	—	60:B	kraj Ds	brisanje nepostojećeg → greška u Dg

```

Dn (nova sekvencijalna):  10  15  30'  40  50
Dg (datoteka grešaka):    40:D — slog već postoji
                           60:B — slog ne postoji

```

Dužina intervala između dva ažuriranja. Ako je datoteka od 51 bloka, a dnevno stigne 5 promena, ažurirati svaki dan znači 51 pristup za 5 promena (≈ 10 pristupa po promeni). Ako se čeka nedelju dana, isti 51 pristup obradi 35 promena ($\approx 1,5$ pristupa po promeni) — ali je sadržaj datoteke do nedelju dana neusaglašen sa stvarnim stanjem. To je ceo kompromis: duži interval → veća efikasnost, duže vreme neusaglašenosti.

26 Sekvencijalna u redoslednoj obradi

Datoteka ima $B = 51$ blok, treba obraditi $M = 200$ promena.

direktna obrada: svaka promena je zasebno traženje, prosečno 26 pristupa
 $200 \times 26 = 5200$ pristupa

redosledna obrada: promene se prethodno uredi po ključu, pa se svaki blok
 učitava tačno jedanput
 51 pristup

Uslov je da vodeća datoteka generiše **logički naredne** vrednosti ključa — svaki korak je traženje logički narednog sloga, metodom linearnog traženja od tekuće pozicije:

Dp: 15 → 35 → 55 → 95 (uređeno)
 Ds: 10 20 | 30 40 | 50 60 | 70 80 | 90 100
 blok1 blok2 blok3 blok4 blok5

traži 15 → blok1 traži 35 → blok2 (ne vraća se na blok1)
 traži 55 → blok3 traži 95 → blok5

Nijedan blok se ne učitava dva puta i nijedan se ne preskače unazad.

27 Građenje indeksa kod indeks-sekvencijalne datoteke

Primarna zona, faktor blokiranja $f = 3$:

blok 1: [10 | 20 | 30] blok 2: [40 | 50 | 60] blok 3: [70 | 80 | 90]

Indeks je posebna datoteka sa parovima (vrednost ključa, relativna adresa bloka), uz propagaciju najvećih vrednosti:

indeks: (30 → 1) (60 → 2) (90 → 3)

Traženje ključa 50:

1. pristup indeksu: $50 \leq 30?$ ne → $50 \leq 60?$ da → blok 2
 2. pristup bloku 2: [40 | 50 | 60] → nađen

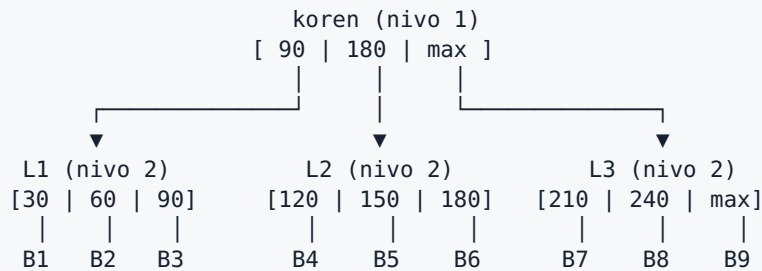
Dva pristupa umesto do tri (ovde) odnosno do B pristupa kod veće datoteke. Kod statičke indeks-sekvencijalne datoteke indeks se formira jednom i posle kreiranja se ne ažurira.

28 Kako se formira zona indeksa

Puno stablo traženja reda $n = 3$, visine $h = 2$, nad primarnom zonom od 9 blokova ($f = 3$ elementa po čvoru).

Primarna zona:
 B1:[10 20 30] B2:[40 50 60] B3:[70 80 90]
 B4:[100 110 120] B5:[130 140 150] B6:[160 170 180]
 B7:[190 200 210] B8:[220 230 240] B9:[250 260 270]

Varijanta 1 — propagacija najvećih vrednosti ključa. Prvo se formiraju svi listovi (nivo $h = 2$), pa se ide naviše do nivoa 1. U svaki element upisuje se najveća vrednost ključa iz odgovarajućeg podređenog bloka odnosno čvora:



U poslednji element krajnjeg desnog čvora upisuje se **maksimalna dozvoljena vrednost ključa**, a ne 270 — inače se ključevi veći od 270 ne bi mogli ni potražiti.

Traženje ključa 140 — traži se prvi element $\geq$ argumenta:

```

koren: 140 ≤ 90? ne → 140 ≤ 180? da → drugi pokazivač → L2
L2:    140 ≤ 120? ne → 140 ≤ 150? da → drugi pokazivač → B5
B5:    [130 | 140 | 150] → nađen
  
```

Ukupno $h + 1 = 3$ pristupa, bez obzira na veličinu datoteke.

Varijanta 2 — propagacija najmanjih vrednosti ključa. Upisuje se najmanja vrednost iz podređenog bloka, a u prvi element krajnjeg levog čvora minimalna dozvoljena vrednost:

```

koren: [min | 100 | 190]
L1: [min | 40 | 70]   L2: [100 | 130 | 160]   L3: [190 | 220 | 250]
  
```

Pravilo traženja se obrće — bira se poslednji element $\leq$ argumenta.

05

Rasute (hash) datoteke

U svim primerima adresni prostor ima $n = 3$ pozicije, dakle adrese 0–999.

29 Metoda centralnih cifara kvadrata ključa

```
k = 172148
k² = 29 634 933 904      ← 11 cifara
    2 9 6 3 4 9 3 3 9 0 4
          ↑ ↑ ↑
    centralne 3 cifre (pozicije 5, 6, 7)

A = 493
```

Drugi ključ, susedan prvom:

```
k = 172149
k² = 29 635 278 201
    2 9 6 3 5 2 7 8 2 0 1
          ↑ ↑ ↑

A = 527
```

Poenta metode: dve gotovo iste vrednosti ključa (172148 i 172149) daju udaljene adrese (493 i 527). Centralne cifre kvadrata zavise od **svih** cifara ključa, jer u množenju učestvuju svi parovi cifara — krajnje cifre kvadrata zavise samo od krajnjih cifara ključa i zato se ne uzimaju.

Ako kvadrat ima paran broj cifara, centralne cifre nisu jednoznačne — usvaja se jedna konvencija (npr. pomeranje ulevo) i primenjuje se dosledno na sve ključeve.

30 Metoda preklapanja kod statički rasutih datoteka

Ključ ima $p = 9$ pozicija, adresa $n = 3$ pozicije:

```
broj segmenata = [p / n] = [9 / 3] = 3

k = 481 273 596 → segmenti: 481 | 273 | 596
```

Cifre se premeštaju kao pri savijanju hartije — spoljni segmenti se preklope preko srednjeg, pa im se redosled cifara obrne:

```
    481      273      596
    ↓ savij  ↓       ↓ savij
    184      273      695

zbir = 184 + 273 + 695 = 1152
A = 1152 mod 10³ = 152
```

Sabiranje je po modulu v^n (v = osnova brojnog sistema = 10, n = 3), pa se odbacuje prenos preko treće pozicije.

Varijanta bez obrtanja cifara (prosto sabiranje segmenata):

$$481 + 273 + 596 = 1350 \rightarrow A = 350$$

Metoda se primenjuje kad je p znatno veće od n — ovde 9 prema 3. Da su segmenti sabrani bez preklapanja i bez modula, vrednosti bi se gomilale u gornjem delu adresnog prostora.

31 Metoda ostatka pri deljenju

Adresni prostor od 997 baketa (997 je prost broj):

$$A = k(S) \bmod m$$

$$k = 172148 \rightarrow 172148 \bmod 997 = 664$$

$$k = 172149 \rightarrow 172149 \bmod 997 = 665$$

$$k = 172150 \rightarrow 172150 \bmod 997 = 666$$

Susedne vrednosti ključa daju susedne adrese, pa se paket bliskih ključeva smešta u susedne bakete — zato je metoda pogodna kad se vrednosti ključa javljaju u paketima (npr. brojevi indeksa jedne generacije).

Zašto $m = 997$, a ne $m = 1000$:

$$k = 172148 \rightarrow 172148 \bmod 1000 = 148$$

$$k = 173145 \rightarrow 173145 \bmod 1000 = 145$$

$$k = 999148 \rightarrow 999148 \bmod 1000 = 148 \quad \leftarrow \text{ista adresa kao prvi ključ}$$

Za $m = 10^3$ adresa je prosto poslednje tri cifre ključa — sve prethodne cifre se gube. Ako su ključevi npr. redom brojevi koji se završavaju nulom, koristi se samo 100 od 1000 baketa. Kod prostog broja u deljenju učestvuju sve cifre ključa.

Kolizije se ne izbegavaju ni ovako: $172148 \bmod 997 = 664$ i $173145 \bmod 997 = 664$ — oba sloga idu u baket 664, pa je potreban postupak za smeštanje prekoračilaca.

32 Smeštanje prekoračilaca kod statički rasute datoteke

Kapacitet baketa je 3 sloga, ključevi k_1 , k_2 , k_3 , k_4 se svi preslikavaju u baket 5:

baket 5 (matični): $[k_1 \mid k_2 \mid k_3] \leftarrow \text{pun}$
 $k_4 \rightarrow \text{prekoračilac}$

a) Otvoreno adresiranje — traži se prvi naredni baket sa slobodnim mestom:

baket 5: $[k_1 \mid k_2 \mid k_3]$
 baket 6: $[k_4 \mid - \mid -] \leftarrow k_4 \text{ smešten ovde}$

Traženje k_4 : čita se baket 5, nije tu, čita se baket $6 \rightarrow 2$ pristupa. Problem: prekoračiocci iz baketa 5 zauzimaju mesto slogovima koji pripadaju baketu 6, pa se lanac širi.

b) Zona prekoračilaca sa ulančavanjem — matični baket nosi pokazivač:

```

primarna zona      zona prekoračilaca
baket 5: [k1|k2|k3] → P1: [k4 | →/]

```

Traženje k4: baket 5, pa P1 → 2 pristupa, ali baket 6 ostaje netaknut.

U oba slučaja upisu prekoračioca prethodi neuspešno traženje u matičnom baketu, a broj pristupa raste sa popunjenošću datoteke — zato se statički adresni prostor dimenzioniše sa rezervom (tipično 70–80 % popunjenosti).

33 Rasuta datoteka i B-stablo

Rasuta datoteka — redosled je nevažan. Slogovi se upisuju hronološki, a mesto određuje isključivo hash funkcija:

```

upisuju se redom: k=250 (A=1), k=118 (A=3), k=907 (A=1), k=442 (A=0)

baket 0: [442]
baket 1: [250 | 907]
baket 2: [ ]
baket 3: [118]

```

Fizička struktura ne sadrži nikakvu informaciju o vezama između slogova — logički naredni slog (npr. 250 → 442) nigde nije zapisan. Zato rasuta datoteka odlično opslužuje traženje slučajno odabranog sloga, a loše redoslednu obradu.

Statička vs. dinamička: kod statičke se broj baketa (npr. 1000) određuje unapred i ne menja tokom eksploatacije — kad se napuni, rastu lanci prekoračilaca. Kod dinamičke se adresni prostor širi tokom ažuriranja.

B-stablo, rang $r = 2$:

```

red  $n = 2r + 1 = 5$           ← maksimalan broj pokazivača iz čvora
maksimalno  $2r = 4$  elementa po čvoru
minimalno  $r = 2$  elementa po čvoru (koren: minimalno 1)

```

Ekstremni slučajevi popunjenosti za $h = 2$:

```

kompletno (popunjeno):      poluprazno:
koren: [10|20|30|40]        koren: [30]
      5 deca × 4 elementa      2 deteta × 2 elementa
ukupno: 4 + 20 = 24 elementa  ukupno: 1 + 4 = 5 elemenata

```

Stablo iste visine ne može imati manje od 5 ni više od 24 elementa.

Upis elementa 25 u pun čvor (prethodi mu neuspešno traženje):

```
pre:    [10 | 20 | 30 | 40]      ← već ima  $2r = 4$  elementa, nema mesta
sortirano sa novim: 10, 20, 25, 30, 40
razdvajanje (cepanje):

          [25]                    ← srednji element se propagira naviše
         /   \
    [10 | 20] [30 | 40]          ← oba nova čvora imaju po  $r = 2$  elementa
```

Brisanje — prethodi mu uspešno traženje; fizičko brisanje elementa iz B-stabla moguće je samo ako se element nalazi u listu, dok se slog u primarnoj zoni briše logički.

06

Katalog i sistemske tabele datoteka

34 Katalog i upravljanje katalogom

Hijerarhijska struktura direktorijuma je stablo — koren se formira automatski, a svaki čvor je direktorijum ili datoteka:



Referenciranje za tekući direktorijum `/home/student/baze`:

način	zapis	rezultat
apsolutno	<code>/home/student/baze/ispit.txt</code>	polazi od korena, uvek isti čvor
relativno	<code>ispit.txt</code>	polazi od tekućeg direktorijuma
relativno naviše	<code>../os</code>	<code>/home/student/os</code>

Iste rutine za upravljanje katalogom u praksi:

<code>mkdir baze</code>	kreiranje direktorijuma
<code>rmdir baze</code>	brisanje direktorijuma
<code>mv a.txt b.txt</code>	preimenovanje / premeštanje datoteke u strukturi
<code>ls</code>	izlistavanje sadržaja direktorijuma
<code>chmod 640 a.txt</code>	dodela i ukidanje prava pristupa
<code>ln stara nova</code>	povezivanje (link) – jedna datoteka, dva zapisa u katalogu

35 Sistemska i alokaciona tabela datoteka

STD — trajan zapis o datoteci koji održava OS, nastaje pri `create`, nestaje pri brisanju datoteke:

STD za `/home/student/baze/radnici.dat`

naziv	radnici.dat
verzija	3
vrsta	binarna, slogovi 200 B
veličina	10 240 B
vlasnik	student, prava 640
pokazivač na ATD	→

ATD — mapa alociranog prostora, neprazan niz parova (pokazivač, broj blokova):

ATD: (120, 8) (350, 4) (900, 16)

```

      |
      |
      |└─ veličina zone: 8 blokova
      └─ adresa početka alocirane zone
  
```

ukupno alocirano = 8 + 4 + 16 = 28 blokova

Datoteka nije u jednom komadu na disku — leži u tri zone, a ATD ih drži na okupu.

alociranje nove zone → ATD: (120,8) (350,4) (900,16) (1400,8) ← dodat par
 delociranje zone 350 → ATD: (120,8) (900,16) (1400,8) ← obrisan par

36 Tabela otvorenih datoteka (TOD)

TOD je niz zapisa o **svim** otvorenim datotekama u sistemu — indeks niza je redni broj otvorene datoteke u sistemu. Svako otvaranje je jedan zapis, pa ista datoteka otvorena dvaput daje dva zapisa:

TOD

i	način korišćenja	tekući pokazivač	baferi	→ TOS
0	O_RDONLY	4096	B7	→ TOS(A)
1	O_RDWR	0	B2, B3	→ TOS(A)
2	O_APPEND	10240	B9	→ TOS(B)

proces P1 čita radnici.dat
 proces P2 piše u istu dat.
 proces P1 piše u log.txt

Zapisi 0 i 1 pokazuju na **isti** TOS (jedna datoteka na disku, jedan opis), ali svaki ima **svoj** tekući pokazivač — zato P1 i P2 čitaju sa različitih pozicija nezavisno.

Veza tabela pri jednoj **read** operaciji:

TP (proces) → TOP/TLI → TOD → TOS (= STD + ATD u OM) → TU → disk
 fd = 3 zapis 0 pozicija adrese blokova drajver

07

Sistemske pozivi za rad sa datotekama

38 Sistemske pozivi

Pogled na datoteku kao niz bajtova. Program vidi neprekinut niz, OS vidi blokove:

```
program:  bajt 0  _____ bajt 10 239
disk:     [ blok 0 ][ blok 1 ][ blok 2 ] (po 4096 B)
```

Transformacija rednog broja bajta u redni broj bloka (blok = 4096 B):

```
bajt 2 000  → blok 2000 div 4096 = 0,  pomak u bloku 2000
bajt 10 000 → blok 10000 div 4096 = 2,  pomak u bloku 1808
```

Zatim se redni broj bloka preko ATD prevodi u apsolutnu adresu na disku — aplikativni program o tome ne zna ništa, i to je upravo nezavisnost od fizičkih karakteristika uređaja.

Izuzeci (događaj koji prekida normalan tok obrade):

```
open("nepostojeca.dat", O_RDONLY) → greška: datoteka ne postoji
read(fd, &s, 200) na kraju dat.   → vraća 0 pročitanih bajtova
write(fd, &s, 200) na punom disku → greška pri zapisivanju
```

40-45 Create, Open, Read, Write, Seek, Close — zajednički primer

Zadatak: povećati platu 11. radniku za 10 %, u datoteci sa slogovima konstantne dužine od 200 B.

```

struct Slog { int mbr; char ime[30]; double plata; /* ... ukupno 200 B */ };
struct Slog s;

/* Create – kreira novu datoteku i otvara je za pisanje */
int fd = create("/home/student/radnici.dat", 0640);
close(fd);

/* Open – priprema datoteke za operativnu upotrebu */
fd = open("/home/student/radnici.dat", 0_RDWR);

/* Seek – direktan pristup: pozicioniranje na početak 11. sloga */
seek(fd, 10 * sizeof(s), SEEK_SET);

/* Read – prenos 200 B iz datoteke u promenljivu s */
read(fd, &s, sizeof(s));

s.plata = s.plata * 1.10;

/* Seek – povratak na početak istog sloga (offset < 0, od tekuće pozicije) */
seek(fd, -(int) sizeof(s), SEEK_CURR);

/* Write – prenos 200 B iz promenljive s u datoteku */
write(fd, &s, sizeof(s));

/* Close – uredan prestanak upotrebe */
close(fd);

```

Šta se dešava sa tekućim pokazivačem:

poziv	tekući pokazivač pre	posle	napomena
<code>open(..., 0_RDWR)</code>	—	0	inicijalizacija na početak datoteke
<code>seek(fd, 2000, SEEK_SET)</code>	0	2000	referentna tačka je početak
<code>read(fd, &s, 200)</code>	2000	2200	automatski se uvećava za broj prenetih bajtova
<code>seek(fd, -200, SEEK_CURR)</code>	2200	2000	offset < 0 → prema početku datoteke
<code>write(fd, &s, 200)</code>	2000	2200	prepisuje isti slog
<code>close(fd)</code>	2200	—	zapis se uklanja iz TOD

Šta radi svaki poziv „ispod”:

- **Create** — proverava volumen, putanju i raspoloživ prostor, alokira inicijalni prostor, kreira STD i ATD, formira zapis u direktorijumu i uvezuje ga sa STD, vraća fajl deskriptor.
- **Open** — proverava uslove i ovlašćenja, prenosi sadržaj STD u OM (kreira TOS), formira zapis u TOD i uvezuje ga sa TOS i TLI, inicijalizuje tekući pokazivač (na 0, ili na kraj ako je `O_APPEND`), rezerviše sistemske bafere i puni ih početnim blokovima.
- **Read** — proverava da li je fd otvoren i da li dozvoljava čitanje, računa redni broj prvog bloka (2000 → blok 0, pomak 2000), proverava da li je blok već u baferu, ako nije računa apsolutnu adresu i inicira fizički prenos, prenosi traženi sadržaj iz bafera u `s`, uvećava tekući pokazivač, vraća broj stvarno pročitanih bajtova.
- **Write** — isto, u suprotnom smeru; u režimu `O_RDWR` ciljani blok se **prvo mora učitati** (menja se samo 200 od 4096 bajtova bloka), dok kod `O_APPEND` ili `O_WRONLY` to najčešće nije potrebno.

- **Seek** — samo menja vrednost tekućeg pokazivača, bez ijednog pristupa disku: `SEEK_SET` (0) od početka, `SEEK_CURR` (1) od tekuće pozicije, `SEEK_END` (2) od kraja.
- **Close** — prazni izmenjene bafere na disk, prenosi ažurirani TOS u STD, oslobađa bafere i briše zapis iz TOD. Dešava se i automatski, završetkom programa.

Zašto Seek mora pre Write. `read` je pomerio pokazivač na kraj sloga (2200). Bez povratka na 2000, `write` bi upisao izmenjeni slog preko **12.** radnika.

Primeri načina otvaranja:

```
open("log.txt", O_WRONLY | O_APPEND); /* pokazivač na kraj, stari sadržaj ostaje */
open("log.txt", O_WRONLY | O_TRUNC);  /* sadržaj se prethodno briše */
open("nova.dat", O_RDWR | O_CREATE); /* ako ne postoji, sprovodi se poziv Create */
open("radnici.dat", O_RDONLY);        /* write nad ovim fd vraća grešku */
```

Sitne ispravke uz skriptu

Tri mesta koja vredi proveriti u slajdovima pre učenja napamet:

1. **Pitanje 16** — „Projekcija ... naziva se još i restrikcija”. Restrikcija je drugi naziv za **selekciju** (izbor vrsta), dok je projekcija izbor kolona.
2. **Pitanje 20** — „Nije nemoguće da kapacitet bloka bude manji ili jednak kapacitetu fizičkog bloka” deluje kao omaška: pošto je blok niz od 2^n ($n \geq 0$) fizičkih blokova, njegov kapacitet ne može biti **manji** od kapaciteta fizičkog bloka.
3. **Pitanje 07** — „posebnu vrstu tipa (entiteta ili poveznika – 18erund)”: reč je o **gerundu**, „18” je artefakt izvoza iz Notion-a.